

File System - Introduction

Overview

This document provides a comprehensive guide to working with files in Node.js, focusing on reading and manipulating files using the built-in `fs` (File System) module. The lecture covers key concepts including binary data, buffers, character encodings, and synchronous file operations. Additionally, an application is introduced that demonstrates file system operations such as creating, deleting, renaming, and modifying files dynamically.

Prerequisites

Before diving into file operations, it's essential to understand:

- Binary data buffers
- Character encodings
- Basic Node.js operations
- Working with terminal and code editors

Setting Up the Environment

1. Create a project folder:

```
mkdir app  
cd app
```

2. Create a JavaScript file:

```
touch app.js
```

3. Open the folder in a code editor:

Open `app` in your preferred editor (e.g., VS Code).

4. Create a text file:

```
touch text.txt
```

Add some sample text inside `text.txt` :

```
Some random text
```

Reading a File in Node.js

To read and display file contents, follow these steps:

1. Import the `fs` module:

```
const fs = require('fs');
```

2. Read the file synchronously:

```
const contents = fs.readFileSync('text.txt');  
console.log(contents);
```

3. Run the script:

```
node app.js
```

Output:

```
<Buffer 53 6f 6d 65 20 72 61 6e 64 6f 6d 20 74 65 78 74>
```

The content is displayed as a buffer (binary format).

4. Convert buffer to string:

```
console.log(contents.toString('utf-8'));
```

Output:

```
Some random text
```

Understanding Buffers and Encoding

- File content is read as binary data (0s and 1s) and stored in a buffer.
- Each character is represented in hexadecimal format.
- UTF-8 encoding is used to map bytes to characters.

Building a File Watching Application

This application continuously monitors a `command.txt` file and executes commands like creating, deleting, renaming, and appending files based on the file's content.

Features

- Watch for file changes
- Execute file operations based on commands
- Modify files dynamically

Commands Supported

Command	Description
create file <filepath>	Creates a file at the specified path
delete file <filepath>	Deletes the specified file
rename <old_filepath> <new_filepath>	Renames a file
add to file <filepath> <content>	Appends content to a file

Example Usage

1. Start the application:

```
node app.js
```

2. Modify `command.txt` with a command:

```
create file /path/to/app/test.txt
```

On saving the file, `test.txt` is created.

3. Rename a file:

```
rename /path/to/app/test.txt /path/to/app/newname.txt
```

4. Append text to a file:

```
add to file /path/to/app/newname.txt "Hello, this is appended."
```

5. Delete a file:

```
delete file /path/to/app/newname.txt
```

Implementation Notes

- The application **watches** `command.txt` for changes.
- When a command is detected, the respective file operation is executed.
- Only the **Node.js** `fs` **module** is used (no third-party libraries).

Next Steps

Before proceeding with further development, it's important to understand:

- What a file represents in an operating system.
- How file systems manage storage and retrieval.
- Operating system-level file handling mechanisms.

This discussion will be covered in the next section.

Summary

- Learned how to read files using Node.js `fs` module.
- Understood binary data, buffers, and character encoding.
- Built an application to watch and manipulate files dynamically.
- Demonstrated real-world use cases for the file system module.

Stay tuned for the next discussion on **understanding files in operating systems!**

File System: What Exactly Is a File?

Introduction

A file is a fundamental concept in computing. It is a sequence of bits, the meaning of which is defined by the user. The interpretation of these bits depends on how the file is opened and

decoded. This document provides an in-depth understanding of what a file is, how files are structured on a storage device, and how NodeJS interacts with files.

Understanding a File

- A file is a **sequence of bits** with a meaning assigned by the user.
- The meaning of the bits in a file depends on how the file is interpreted:
 - **Text File:** Bits represent characters (e.g., 'C').
 - **Image File:** The same sequence of bits may represent a color code or pixel location.
 - **Video File:** The bits may represent frames, audio data, and timing information.
- **Decoding Mechanism:** Different types of files require different decoding mechanisms:
 - Character decoding for text files.
 - Image decoding for images.
 - Video decoding for video files.

Files and Storage Devices

- Everything on a computer is a file:
 - Hard drives (HDDs, SSDs)
 - Flash drives, USB drives
 - Programs, data files (text, image, video, etc.)
- A file can represent different things:
 - Programs (executables, scripts, applications)
 - Data (images, videos, text files)
- A **storage unit** on a storage device is always a file.

File Metadata

Each file has additional metadata stored in another file (often a **file table** or **file descriptor**):

- **File Name:** The name assigned to the file.
- **Protection:**
 - Whether a file can be read, written, or executed.
 - Whether other users can access the file.
- **Location:** The path of the file on the storage device.
- **File Type:**
 - Determines how the file should be interpreted.

- Example: `.txt` (text file), `.jpg` (image file), `.mp4` (video file), `.exe` (executable file).
- **Timestamps:**
 - Last accessed time.
 - Last modified time.
 - Creation time.
- **Other Metadata:** Additional system-related information.

Hard Drive and File Structure

- The entire hard drive is divided into files.
- Every sequence of bits on the hard drive belongs to some file.
- Data on a hard drive exists in the form of files.
- Even the **operating system itself** is a collection of files.

Examples of Files in an Operating System

- Running commands in a terminal:
 - Commands such as `ls`, `mkdir`, and `which` are files.
 - Example:

```
which mkdir
```

 - This command shows the path of the `mkdir` executable file.
 - The `mkdir` file is stored in the `/bin/` directory.
- Inspecting a file's properties:
 - Example: Right-click on a file and select **Get Info**.
 - Displays:
 - File type (e.g., UNIX executable, text file, image file).
 - File size.
 - Location.
 - Timestamps (creation, modification, access times).

Operating System and File Execution

- The **operating system** itself is a collection of executable files.
- The OS can **only execute** executable files.
- It does **not inherently know** how to open image, video, or text files.

- Other applications provide decoding mechanisms:
 - **macOS Preview**: Opens image files.
 - **PDF Reader**: Opens .pdf files.
 - **Media Player**: Opens video and audio files.
- Each **executable file** is specific to an **operating system** and **CPU architecture**.

Application Files

- Applications consist of multiple files.
- Example: macOS Calendar App
 - Right-click → **Show Package Contents**.
 - Displays multiple files and folders used by the application.
 - Includes:
 - **Executable files**.
 - **Configuration files**.
 - **Data files**.

NodeJS and File Handling

- NodeJS is a **server-side technology** that must interact with files.
- File operations in NodeJS include:
 - **Deleting a file**.
 - **Appending data to a file**.
 - **Renaming a file**.
- NodeJS provides built-in modules for file handling:
 - **fs (File System) module** for interacting with the filesystem.
 - Allows synchronous and asynchronous file operations.

Conclusion

- Everything on a computer is a file, including the OS itself.
- Files are sequences of bits interpreted based on their type.
- NodeJS, being a server-side technology, provides mechanisms to interact with files efficiently.
- Understanding files and their structure is crucial for file manipulation in any programming environment.

This document provides a comprehensive understanding of files, storage, metadata, operating system interaction, and how NodeJS handles files.

Node.js File System: How Node.js Deals with Files

Introduction

In the previous discussion, we explored the concept of files and their role in the operating system. Now, in this document, we will delve into how Node.js interacts with these files.

Node.js and File System

Understanding the Hierarchy:

- Our **hard drive** stores the files.
- The **operating system** runs on top of the hard drive and manages hardware resources.
- Node.js runs as a **process** within the operating system.
- **Processes** in the OS facilitate interactions between software and hardware.
- **Node.js does not directly access the hard drive**; instead, it interacts with the operating system.

How Node.js Accesses Files

Node.js does not have direct access to the hard drive to modify bits. Instead, it interacts with the operating system through **system calls**. These system calls are responsible for performing operations like reading, writing, and modifying files.

System Calls in Node.js

- **System calls** are special functions that allow Node.js to interact with the OS.
- When you open a file in Node.js, you are not directly manipulating the file on the hard drive.
- Instead, Node.js makes a request to the operating system via a system call.
- The OS then retrieves the file data (in binary: 0s and 1s) and provides it back to Node.js.

The open System Call

- If you want to **open** a file in Node.js, you use an `open` system call.
- Node.js makes a request via **Libuv**, which then calls the `open` system call.
- The operating system fetches the file from the hard drive and returns the data to Node.js.

File Operations in Node.js

Besides opening files, Node.js allows various file operations through system calls, including:

1. **Read a file** (fetching the content from the hard drive)
2. **Write a file** (modifying or creating new files)
3. **Rename a file** (changing the filename on the disk)

How Libuv Helps

- **Libuv** is a library that handles asynchronous I/O operations in Node.js.
- It abstracts system calls and makes file operations seamless.
- Instead of directly calling system calls, Node.js interacts with Libuv, which in turn interacts with the operating system.
- This ensures efficient management of file operations.

Thread Pool in Libuv

- **Libuv uses a thread pool** to manage file operations.
- When performing file operations, Node.js may need multiple threads for efficiency.
- If file operations are heavy, they may **exhaust the thread pool**.
- Managing these threads properly ensures smooth performance.
- More details on thread pools will be discussed later in the course.

Summary

- Node.js does not directly manipulate files on the hard drive.
- It interacts with the **operating system** via **system calls**.
- **Libuv** acts as a middleman between Node.js and system calls.
- **Thread pools** in Libuv help manage file operations efficiently.
- We can perform operations like opening, reading, writing, and renaming files in Node.js.
- Understanding how these processes work gives better insight into file handling in Node.js.

Next Steps

In the upcoming lessons, we will move to **practical coding** in Node.js to see how file operations work in real-world applications.

Stay tuned for the next session!

File System: Three Different Ways of Doing the Same Thing in Node.js

Overview

In Node.js, the `fs` (File System) module allows us to work with files and directories. When performing file operations, there are three different approaches available:

1. **Promises API**
2. **Callback API**
3. **Synchronous API**

Each of these approaches achieves the same outcome but differs in how they handle execution and performance.

Three Ways to Handle File System Operations

1. Promises API

The Promises API is the recommended approach, as it provides cleaner and more readable code, especially with `async/await`.

Example:

```
const fs = require('fs/promises');

async function copyFile() {
  try {
    await fs.copyFile('file.txt', 'copied-promise.txt');
    console.log('File copied successfully using Promises API');
  } catch (error) {
    console.error('Error copying file:', error);
  }
}

copyFile();
```

- Uses `fs/promises` instead of `fs` .
- Uses `async/await` for better readability.
- Error handling with `try/catch` .

Recommended in **90-99% of cases**.

2. Callback API

The Callback API uses traditional callback functions to handle errors and responses. It is slightly more complex than Promises API and can lead to callback nesting.

Example:

```
const fs = require('fs');

fs.copyFile('file.txt', 'copied-callback.txt', (err) => {
  if (err) {
    console.error('Error copying file:', err);
    return;
  }
  console.log('File copied successfully using Callback API');
});
```

- Uses `fs.copyFile` with a callback function.
- The first argument of the callback is always an error (error-first callback pattern).
- Can be more performant in highly CPU-intensive applications.

Use only when **maximum performance** is needed.

3. Synchronous API

The Synchronous API is blocking and should be avoided unless necessary. It should only be used when operations **must complete before proceeding**, such as reading configuration files at startup.

Example:

```
const fs = require('fs');

try {
  fs.copyFileSync('file.txt', 'copied-sync.txt');
  console.log('File copied successfully using Synchronous API');
} catch (error) {
  console.error('Error copying file:', error);
}
```

- Uses `fs.copyFileSync` which blocks the main thread.
- No callbacks or promises.
- If an error occurs, the process terminates unless handled with `try/catch`.

Avoid unless absolutely necessary (e.g., reading configuration before application starts).

Differences Between the Three Approaches

Approach	Blocking	Performance	Readability	Error Handling
Promises API	Non-blocking	Good	Best (async/await)	Try/Catch
Callback API	Non-blocking	Best (for performance-critical tasks)	Less readable	Callback function
Synchronous API	Blocking	Worst (blocks main thread)	Simple but risky	Try/Catch needed

Best Practice: Stick with the **Promises API** unless performance demands the Callback API or the situation mandates Synchronous API.

Example: Copying a File Using All Three Methods

```
const fs = require('fs');
const fsPromises = require('fs/promises');

// Using Promises API
async function copyFilePromise() {
  try {
    await fsPromises.copyFile('file.txt', 'copied-promise.txt');
    console.log('Copied using Promises API');
  } catch (err) {
    console.error(err);
  }
}

// Using Callback API
function copyFileCallback() {
  fs.copyFile('file.txt', 'copied-callback.txt', (err) => {
    if (err) {
      console.error(err);
      return;
    }
    console.log('Copied using Callback API');
  });
}

// Using Synchronous API
function copyFileSync() {
  try {
    fs.copyFileSync('file.txt', 'copied-sync.txt');
    console.log('Copied using Synchronous API');
  } catch (err) {
    console.error(err);
  }
}

// Execute the functions
copyFilePromise();
copyFileCallback();
copyFileSync();
```

Error Handling

If an error occurs (e.g., the source file does not exist), here's how each API reacts:

- **Promises API:** Returns an error object inside `catch`.
- **Callback API:** The error is passed as the first argument of the callback.
- **Synchronous API:** The entire application crashes unless handled with `try/catch`.

Node.js Documentation Reference

To explore more file operations in Node.js:

- [Node.js File System \(fs\) Module Documentation](#)

Node.js provides all three APIs in the documentation for every file operation.

Conclusion

- **Use Promises API** (`fs.promises`) for most use cases (cleaner, modern, `async/await` support).
- **Use Callback API** (`fs`) when absolute maximum performance is needed.
- **Avoid Synchronous API** (`fs.sync`) unless it's **absolutely required** (e.g., loading configuration files at startup).

Understanding and choosing the right approach ensures efficiency and maintainability in your Node.js applications.

Happy Coding! 🚀

File System: Watching the Command File for Changes

Overview

This application watches a command file (`command.txt`) for changes and executes commands written in it. The application is built using Node.js and utilizes the `fs.promises.watch` API to monitor file changes in real-time.

Getting Started

Step 1: Create Project Directory

1. Create an empty directory for your project.
2. Open the directory in your preferred code editor.

Step 2: Create Required Files

- `app.js` : This file contains the Node.js code.
- `command.txt` : This file stores commands that the application will watch for changes.

Implementation Details

Step 3: Watching a File for Changes

Node.js provides the `fs.promises.watch` API for watching file changes. To use it:

1. Import the File System (FS) Module

```
const fs = require('fs/promises');
```

2. Create an Immediately Invoked Function Expression (IIFE)
 - This ensures we can use `async/await` and limit scope.


```
(async () => {
  const watcher = fs.watch('./command.txt');
  for await (const event of watcher) {
    console.log(event);
  }
})();
```

Step 4: Understanding Async Iterators

- The `fs.watch` function returns an **async iterator**.
- We loop through it using `for await`.
- This ensures that we capture each event as it occurs.

Step 5: Event Handling

- When a file change is detected, it logs the event object:

```
{
  "eventType": "change",
  "filename": "command.txt"
}
```

- If a file is renamed, created, or deleted, the event type is `rename`.

Step 6: Filtering for the `command.txt` File

- We only care about `change` events for `command.txt`.
- Modify the watcher as follows:

```
(async () => {
  const watcher = fs.watch('./command.txt');
  for await (const event of watcher) {
    if (event.eventType === 'change') {
      console.log('The file was changed');
    }
  }
})();
```

Step 7: Running the Application

1. Open a terminal in your project directory.

2. Run the following command:

```
node app.js
```

3. Modify `command.txt` and save the file to trigger a change event.

Step 8: Handling Multiple Change Events

- Sometimes, changes are detected multiple times.
- This behavior varies by OS and text editor.
- It is due to how the file system handles save operations.
- A workaround is checking the **modification time (mtime)** of the file.

Alternative: Watching a Directory

- Instead of a single file, you can watch an entire directory:

```
(async () => {  
  const watcher = fs.watch('./');  
  for await (const event of watcher) {  
    console.log(event);  
  }  
})();
```

- This logs events for all files in the directory.

Key Takeaways

- Use `fs.promises.watch` to monitor file changes.
- Utilize `async/await` with an **async iterator** to handle events.
- Filter for `change` events to avoid unnecessary processing.
- Be aware of multiple change events and consider workarounds.
- You can watch either a single file or an entire directory.

Conclusion

This guide covers watching a file (`command.txt`) for changes in Node.js. The next steps involve reading file contents and processing commands dynamically.

File System: Reading the Content of the Command File

Overview

This document provides a detailed step-by-step explanation of how to read the contents of a file in Node.js, specifically focusing on opening, reading, and closing a file. The document follows the concepts discussed in the lecture on "File System: Reading the Content of the Command File."

Concept of Reading a File

When reading a file in any programming language, the process follows these general steps:

1. **Opening the file** - A file must be opened before reading or writing.
2. **Reading from the file** - The contents are read and stored in memory.
3. **Writing to the file (if required)** - Data can be written to the file.
4. **Appending to the file (if required)** - New data can be appended to the file.
5. **Getting the status of the file** - File metadata such as last modified time can be accessed.
6. **Closing the file** - The file must be closed to free up system resources.

This is a common approach across different programming languages, including Python, C++, and JavaScript.

Understanding File Descriptors

- When a file is opened, a **file descriptor** is assigned.
- A **file descriptor** is a unique number representing the opened file.
- The file descriptor allows reading, writing, and performing other operations on the file.

Steps to Read a File in Node.js

1. Using the `fs` Module

In Node.js, the `fs` module provides a method `fs.open()` to open a file. It accepts:

- **File path:** The path of the file to be opened.

- **Flag:** Specifies the operation mode ('r' for reading, 'w' for writing, etc.).

```
const fs = require('fs/promises');
async function readFile() {
  const fileHandler = await fs.open('command.txt', 'r');
  console.log('File opened successfully');
}
readFile();
```

2. Understanding the File Handle

- `fs.open()` does **not** read the file; it merely assigns a **file descriptor**.
- The `fileHandler` object contains methods to read, write, or close the file.

3. Closing the File

- It is essential to **close** the file after reading/writing to avoid memory leaks.
- Open files consume system resources, and not closing them can lead to performance issues.

```
async function readFile() {
  const fileHandler = await fs.open('command.txt', 'r');
  console.log('File opened successfully');
  await fileHandler.close();
}
readFile();
```

4. Reading the File Contents

- The method `fileHandler.read()` is used to read data from the file.
- The method returns an object containing:
 - **bytesRead:** Number of bytes read from the file.
 - **Buffer:** Data stored as a buffer.

```

async function readFile() {
  const fileHandler = await fs.open('command.txt', 'r');
  const buffer = Buffer.alloc(1024); // Allocate a buffer of appropriate size
  const { bytesRead } = await fileHandler.read(buffer, 0, buffer.length, 0);
  console.log('Bytes Read:', bytesRead);
  console.log('Data:', buffer.toString('utf8', 0, bytesRead));
  await fileHandler.close();
}
readFile();

```

5. Allocating the Correct Buffer Size

- Instead of using a fixed-size buffer, determine the file size dynamically:

```

async function readFile() {
  const fileHandler = await fs.open('command.txt', 'r');
  const { size } = await fileHandler.stat(); // Get file size
  const buffer = Buffer.alloc(size); // Allocate buffer of correct size
  const { bytesRead } = await fileHandler.read(buffer, 0, size, 0);
  console.log('Data:', buffer.toString('utf8', 0, bytesRead));
  await fileHandler.close();
}
readFile();

```

6. Handling File Positioning

- The file read position must be managed properly.
- If not reset, subsequent reads may return empty data.

```

async function readFile() {
  const fileHandler = await fs.open('command.txt', 'r');
  const { size } = await fileHandler.stat();
  const buffer = Buffer.alloc(size);
  await fileHandler.read(buffer, 0, size, 0); // Ensure position is reset to 0
  console.log('Data:', buffer.toString('utf8'));
  await fileHandler.close();
}
readFile();

```

7. Understanding the Buffer

- Buffers store binary data.
- Each character occupies one byte in UTF-8 encoding.
- The buffer size should match the file size to avoid excess memory allocation.

8. Reading File Metadata

- The `stat()` method retrieves file metadata, including:
 - File size
 - Creation date
 - Last modified time

```
async function getFileStats() {  
  const fileHandler = await fs.open('command.txt', 'r');  
  const stats = await fileHandler.stat();  
  console.log('File Size:', stats.size);  
  console.log('Created:', stats.birthtime);  
  console.log('Last Modified:', stats.mtime);  
  await fileHandler.close();  
}  
getFileStats();
```

Summary

Step	Action
1	Open the file using <code>fs.open()</code>
2	Store file descriptor in <code>fileHandler</code>
3	Use <code>fileHandler.read()</code> to read file contents
4	Allocate buffer dynamically using <code>Buffer.alloc(size)</code>
5	Ensure correct file positioning when reading
6	Close the file using <code>fileHandler.close()</code>
7	Retrieve file metadata using <code>fileHandler.stat()</code>

Conclusion

- The process of reading files in Node.js involves opening, reading, and closing files correctly.
- The `fs` module provides the necessary methods to efficiently read and manage files.
- Proper memory management, such as allocating the correct buffer size and closing files after use, prevents resource leaks.
- The concepts are similar across programming languages, including Python, C++, and JavaScript.

By following these best practices, you can efficiently read and handle file contents in a Node.js environment.

File System: Cleaning Up the Code Using Event Emitter

Overview

This document provides a detailed breakdown of the process of refactoring messy code by leveraging the `EventEmitter` class to clean up file handling operations. This approach makes the code more structured, modular, and maintainable. The step-by-step process below is based on the transcript of a lecture.

Step 1: Removing Redundant Code

The first step is to clean up unnecessary lines of code and replace them with well-documented comments.

Removed Code

Three lines were removed as they were redundant.

Added Comments

Instead of the removed lines, clear comments were added to explain the purpose of different sections of the code:

1. **Get the file size:** Determines the total size of the file.

2. **Allocate buffer:** Allocates memory based on the file size.
3. **Define offset:** Specifies the starting location for filling the buffer, always set to `0`.
4. **Determine bytes to read:** Specifies how many bytes to read.
5. **Set the reading position:** Determines where to start reading the file from.
6. **Read full content:** Ensures that the entire file content is read from beginning to end.

Step 2: Leveraging the EventEmitter Object

An important optimization is recognizing that the `FileHandle` object is an `EventEmitter`.

Understanding EventEmitter

- According to the documentation, all `FileHandle` objects inherit from the `EventEmitter` class.
- This means we can emit and listen to events to streamline our file operations.

Implementing a Custom Event

To clean up the code further, a custom event is introduced:

1. Define the event listener:

```
commandFileHandler.on('change', function () {  
  // Function to handle the change event  
});
```

- The event is named `change` for clarity.
- A function is passed as the event handler.

2. Emitting the Event:

```
commandFileHandler.emit('change');
```

- This triggers the event when necessary.
- The event will be emitted at specific places in the code.

3. Refactoring the Code:

- The old logic is moved into the event handler function.
- The function handling the event is marked `async` to accommodate asynchronous file operations.

This results in a more modular structure:

- One part of the code emits the event.
- Another part listens to and processes the event.

Step 3: Running the Code

Once the event-driven refactoring is complete, the application is executed:

1. **Save the file.**
2. **Run the application in the terminal.**
3. **Observe the output.**

Expected Output

The output remains the same as before, indicating that the refactored approach is functionally equivalent but significantly cleaner.

Step 4: Decoding the Data

Currently, the file contents are being read, but they are represented as binary data (zeros and ones). The next step is to:

1. **Pass the binary data through a decoder.**
2. **Transform the data into a meaningful format.**

This will be covered in the next phase of development.

Conclusion

By using the `EventEmitter` class, the code has been successfully cleaned up, resulting in:

- A **modular** and **structured** approach.
- **Separation of concerns** between event emission and event handling.
- **Better maintainability** and **readability**.

This approach ensures that file handling operations are streamlined and easier to manage.

Next Steps

- Implement a decoder to transform binary data into meaningful information.
- Further optimize the file handling process.

Stay tuned for the next phase!